Introduction to Architecture Exam

ExamTag: 2023-Jun

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Be sure to circle the answers on the page titled "answers page" or your answers will not be graded. If you choose not to answer a question do not circle a letter. Feel free to detach the answers page from the remainder of the exam and only turn it in.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

1. How do we check that the address is correctly word aligned in our MIPS architecture?
   `2019-Oct-ISA 1-1`
   a. Address is a multiple of 2
   b. Address is a multiple of 4
   c. Address has its last two bits both 11
   d. Address does not point to a byte

   Our 32-bit MIPS architecture has words that are 32 bits, which is 4 bytes. As the memory is byte-addressable, to be word-aligned we need to only point to bytes that are on 4-byte boundaries. E.g., multiples of 4 bytes.

2. Whenever the operating system switches between programs it has to store all of the current program's state (all the data stored inside the processor that the program is using) so that when it switches back it can restore it to where the program was with the data it needs. How many bytes are required to store the state we have used in our MIPS programs?
   `2014-Dec-ISA 1-5`
   a. 31
   b. 32
   c. 124
   d. 128

   We need to store 31 registers (no need to store R0 since it is always 0) plus the PC so we know which instruction to return to, for a total of 32 registers, or 128 bytes.

3. What is the value of R2 after this MIPS code is executed? (ignore branch delay slots)

```
R2 = 4, R12=8
beq     R12, 17, skip
addi    R2, R0, 18
j       done
skip:
        addi    R2, R0, 20
done:
```

   `2022-Aug-ISA 1-2`
   a. 18
   b. 20
   c. 22
   d. Program won't work as intended (Assembly is incorrect)

   There is no beq R, constant, constant instruction! (Remember we don't have space for two immediates in an instruction.) Therefore the program is invalid and won't work.

4. Why do we need both a register read and an ALU operation to write to memory?
   `2013-Dec-ISA 1-2`
   a. We don't
   b. The ALU always does something regardless of whether we use the results
   c. The register provides the write data and the ALU calculates the address
   d. The register provides the address and the ALU proves the write data

   The sw instruction uses the ALU to calculate the address (adding the immediate offset to one of the registers) and uses the other register for the data to write.

5. Which of the following will NOT copy the contents of R2 to R1?
   `2015-Oct-ISA 1-6`
   a. addi R1, R2, 0
   b. sll R1, R2, 0
   c. sw R1, R0(12)
      lw R2, R0(12)
   d. add R1, R2, R0

   The sw/lw combination stores R1 into memory and then loads it into R2. This is equivalent to copying R1 to R2, not R2 to R1.

6. How much space does a branch label take up in the machine code?
   `2015-Aug-ISA 1-5`
   a. 0 words
   b. 1 word
   c. 1 byte
   d. 1 byte per character in the label name

   Branch labels are not part of the code. When the code is assembled, the assembler uses the label to know what constant to put into the jump offset. (E.g., if the label "skip" is 4 instructions after the branch where it is called, then the assembler will use the constant +4 as the offset to get to the instruction.)

2. ISA 2

1. What is the maximum distance (in instructions) that a conditional branch instruction can branch in the MIPS ISA?

Name	Bit Fields					
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Format	op	rs	rt	rd	shmt	funct
I-format	op	rs	rt	**address/immediate (16)**		
J-format	op	**target address (26)**				

2023-Jun-ISA 2-1

a. $[0, 2^{16}]$
b. $[-2^{15}, 2^{15}-1]$
c. $[-2^{16}, 2^{16}]$
d. $[-2^{16}, 2^{16}-1]$

A conditional branch is encoded using the I-format. A total of 16 bits can be used for the address offset meaning the branch can jump $[-2^{15}, 2^{15}-1]$ instructions from the PC+4 location. ***Based on question contributed by student***

2. A designer decides to add a fused multiply-add (FMA) instruction to our MIPS processor. The instruction does the following operation on registers: A=A*B+C. What type of instruction format can we use to encode this new instruction? (Assume Register File is extended to allow three concurrent reads)

2014-Dec-ISA 2-2

a. R
b. I
c. J
d. Need a new format

We need three registers so we can use the R format. Note that A is both a source and the destination, so we would need a third read port on our register file.

3. Why is it good to have consistent instruction encoding (MIPS) as opposed to variable or complex instruction encoding (x86)?

2014-Sep-ISA 2-1

a. Smaller instructions
b. Easier to handle constants
c. Easier to decode instructions
d. All of the above

Consistent instruction encoding makes it easier to decode the instructions because you handle each one the same way. However, it means that even very simple instructions need to have the same number of bits, so it can increase the average instruction size. It doesn't have any impact on the ability to handle constants.

4. What registers does this method need to save and restore when it starts and returns to follow the MIPS calling convention? Hint: `$t*` registers are caller saved, `$s#, $sp, $fp, $ra` registers are callee saved.

```
method3:
        addi    $t0, $zero, 0
        beq     $a0, $zero, end
        sll     $t0, $a0, 2
        add     $t0, $t0, $a1
end:
        addi    $vo, $t0, 0
        jr      $ra
```

2013-Dec-ISA 2-6

a. None
b. $t0
c. $t0, $a0, $a1
d. $t0, $a0, $a1, $ra

This method does not call any other methods so we do not need to save any of the registers it uses.

5. How long will this program take to get to finish? (Assume each instruction takes 1 cycle; e.g., no delay slots.)

```
start:
  jal middle
  finish:
middle:
  jal last
  jr $ra
last:
  jr $ra
```

2014-Dec-ISA 2-4

a. 3 cycles
b. 4 cycles
c. 8 cycles
d. Forever

Since middle does not store $ra on the stack before it calls last, when we return from last we will have the address of the jr in middle stored in $ra, so jr $ra in middle will jump right back to jr $ra in middle, and repeat forever.

6. To make procedure calls faster, a designer decides to change the JAL instruction to increment the stack pointer by 2 words automatically and JR to decrement the stack pointer by 2 words automatically. The designer adds special hardware to the processor to do this in the same time as the regular JAL and JR instructions. What effect does this have?
   `2014-Dec-ISA 2-1`
   a. Procedures that need to save 1 or 2 registers are faster
   b. Procedures that need to save more than 2 registers are not faster
   c. Procedures that save fewer than 2 registers waste space on the stack
   d. All of the above

   For procedures that save 1 or 2 words, this optimization avoids having to change the stack pointer before putting data on the stack and after taking it off, which will speed up the procedures. If you need more than 2, however, you still have to do this. For procedures that don't use 2 positions on the stack, this approach will always take up the memory space, which will waste space.

3. Computer Arithmetic

1. What are the steps required to compute `A - B` with two's complement.
   `2023-Jun-Computer Arithmetic-1`
   a. Invert A, add A and B
   b. Invert A, add A and B and 1
   c. Invert B, add A and B
   d. Invert B, add A and B and 1

   Negating a 2's complement number consists of inverting the bits and adding 1. Thus to negate 'B', we would invert B and add 1. ***Based on question contributed by student***

2. What is the result of the following signed-magnitude addition: 1000+0001?
   `2015-Aug-Computer Arithmetic-2`
   a. -1
   b. 0
   c. 1
   d. 8

   This is -0 + 1 which is 1.

3. For a 7-bit normalized floating-point format: $(-1)^S*(1.FFFF)*(2^{EE})$ where FFFF is unsigned and EE is two's complement, what is the smallest positive value you can represent?
   `2015-Oct-Computer Arithmetic-3`
   a. 0.5
   b. 0.25
   c. 0.125
   d. 0.28125

   (0 0000 10) = 1*1.0*2-2 (=1/4).

4. The number 0110 cannot represent which of the following in fixed point:
   `2017-Jan-Computer Arithmetic-4`
   a. 0.375
   b. 0.625
   c. 0.750
   d. 3.000

   0.625 (base-10) = 0.101 (base-2)

5. A serial multiplier trades off what for what vs. a parallel multiplier? (The serial multiplier is...)
   `2014-Apr-Computer Arithmetic-5`
   a. Lower precision, but faster
   b. Faster, but larger circuit area
   c. Slower, but smaller circuit area
   d. They are the same, just different ways to look at the circuit

   The serial multiplier does each step one after another and therefore only needs one adder and one shifter. The parallel multiplier does all the shifts and adds in parallel, which requires much more circuit area, but operates much faster.

6. You want to compute a two's complement add, but accidentally use an unsigned add instruction instead. What can you say about the result, compare to a signed add?
   `2016-Oct-Computer Arithmetic-1`
   a. Signed and unsigned add is the same in hardware, result is always the same
   b. The output is the same only if the input values are positive
   c. The output is always the same, but the overflow flag may be incorrect
   d. It reduces the instruction latency

   Two's complement numbers use a regular adder, but the detection of overflow is different.

4. Logic

1. What does a latch do?
   2017-Jan-Logic-4
   a. Store a value
   b. Update the stored value on the clock signal
   c. Break the feedback loop in a sequential circuit
   d. All of the above

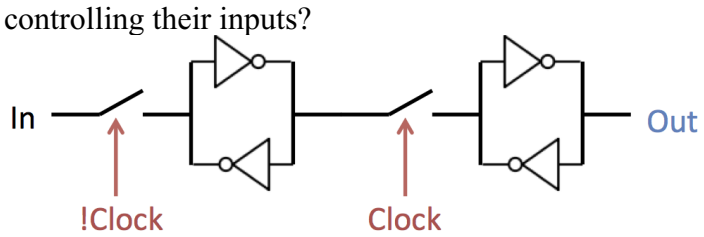
   A latch does all of these things by only transferring its input to the output when the clock changes.

2. As circuits get smaller, the size of the DRAM cells is shrinking. This means there are fewer and fewer electrons to store each bit in the DRAM. What implications does this trend have?

   2016-Jan-Logic-6
   a. The DRAM will become more reliable since it is easier to know whether you read out a 1 or 0 if there are fewer electrons.
   b. The DRAM will become faster since you won't have to wait as long to get all the electrons out to see if it is a 0 or 1.
   c. The DRAM will be more sensitive to cosmic rays that leave random electrons on the chip.
   d. All of the above

   The DRAM will become less reliable since there are fewer electrons and it is harder to know whether you read a 1 or 0. Similarly it will take longer to read as you need to wait longer to get enough electrons. The DRAM will become more sensitive to cosmic rays as there are fewer electrons to hold the value.

3. What is the benefit of having two latches back to back with opposite clock signals controlling their inputs?



   2016-Jan-Logic-5
   a. The data is only updated when the clock goes from low to high
   b. The data only goes through the latch when the clock is low
   c. The latch can store a value when the clock is high
   d. The latch behaves the same as a single latch but is slower

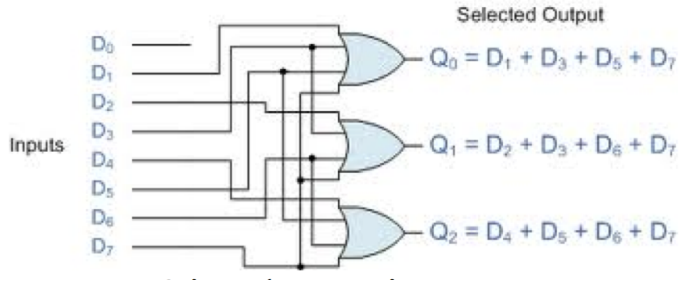
   The second latch causes the output value to only update when the clock changes from low to high. This prevents the data from going straight through the latch at any time. Running more slowly is not a benefit, and the regular latch can also store values when the clock is high.

4. Which is faster, and why? DRAM or SRAM?
   2013-Jan-Logic-5
   a. DRAM: each bit is stored in a powered feedback loop so it can be read quickly
   b. DRAM: each bit is stored in a passive capacitor so it can be read quickly
   c. SRAM: each bit is stored in a powered feedback loop so it can be read quickly
   d. SRAM: each bit is stored in a passive capacitor so it can be read quickly

5. A designer connects a long chain of AND gates together. (E.g., the output from the first AND goes into one of the inputs of the second AND and so forth.) The circuit uses 1.0V to represent 1 and 0.0V to represent 0. When the circuit comes near a mobile phone it experiences +0.1V of noise on the wires between each AND gate. How many AND gates can be put in a chain until the noise can cause the wrong result?
   2015-Aug-Logic-6
   a. 0
   b. 1
   c. 5
   d. No limit

   At each AND gate the inputs are evaluated as 0 if the are < half way (<0.5) and 1 if the are > (>=0.5V). Therefore, the +0.1V that appears on the output will not affect the next AND gate, and each AND gate will eliminate the previous noise, so you can put together as many AND gates as you want. This is why we can build huge digital systems.

6. Why is the D0 input to an encoder not connected to the logic that determines the output?



   2013-Apr-Logic-3
   a. D0 is a dummy input
   b. If D0 is true, then we want the outputs to be all false
   c. D0 is an illegal input
   d. The logic doesn't need D0 as an input because one of the outputs is always true

   If the D0 input is 1, then the encoded output should be 000, so the encoder assigns the output to 000 if none of the other inputs are 1, so it can ignore D0.

5. Processor Control and Datapath

1. What is **not** true about Instruction Fetching (IF) in the Datapath?
   `2023-Jun-Processor Control and Datapath-1`
   a. The IF keeps track of the current instruction address using the PC register
   b. The instruction pointed to by the PC is loaded from the instruction memory
   c. Once the instruction is loaded from instruction memory, it is stored in the register file to be decoded.
   d. The PC+4 value is calculated every cycle

   The PC is used to store the current instruction of this cycle for the IF. Each cycle the value in the PC is used as an address to the Instruction Memory to load in the instruction. This instruction is then decoded (but not stored in the register file!) and executed. Finally, regardless of whether the instruction is a branch or jump instruction, the PC+4 is always calculated (The PC+4 may not be used, if the instruction is a jump or branch, but it is calculated nontheless). ***Based on question contributed by student***

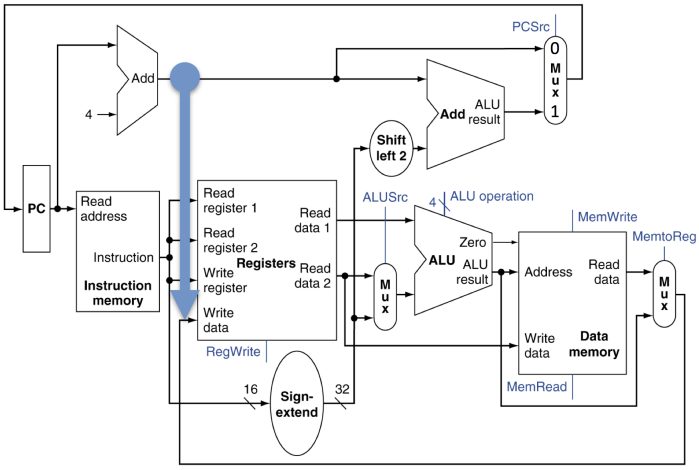
2. A single-cycle processor has three types of instructions: A takes 1ns, B takes 2ns, and C takes 10ns. A program with an instruction mix of 25% A, 70% B, and 5% C will run at what clock speed?
   `2015-Oct-Processor Control and Datapath-5`
   a. 1000 MHz (1/1ns)
   b. 465 MHz (1/2.15ns)
   c. 333 MHz (1/3ns)
   d. 100 MHz (1/10ns)

   The slowest instruction path dictates the clock speed.

3. Why did our single-cycle processor design have two memories?
   `2015-Apr-Processor Control and Datapath-2`
   a. It doesn't, we just treat them as two different memories to understand better
   b. Different parts of the instruction access the instruction memory and the data memory
   c. Some instructions need to access the instruction memory and the data memory at the same time
   d. We don't have a cache so it is faster if we have two memories

   Load/store instructions need to both load the instruction (from the instruction memory) and access data (from the data memory) at the same time.

4. Which instructions need the connection shown in the datapath below with the thick line?



2014-Apr-Processor Control and Datapath-1

a. jr
b. jal
c. jr and jal
d. jr, jal, and j

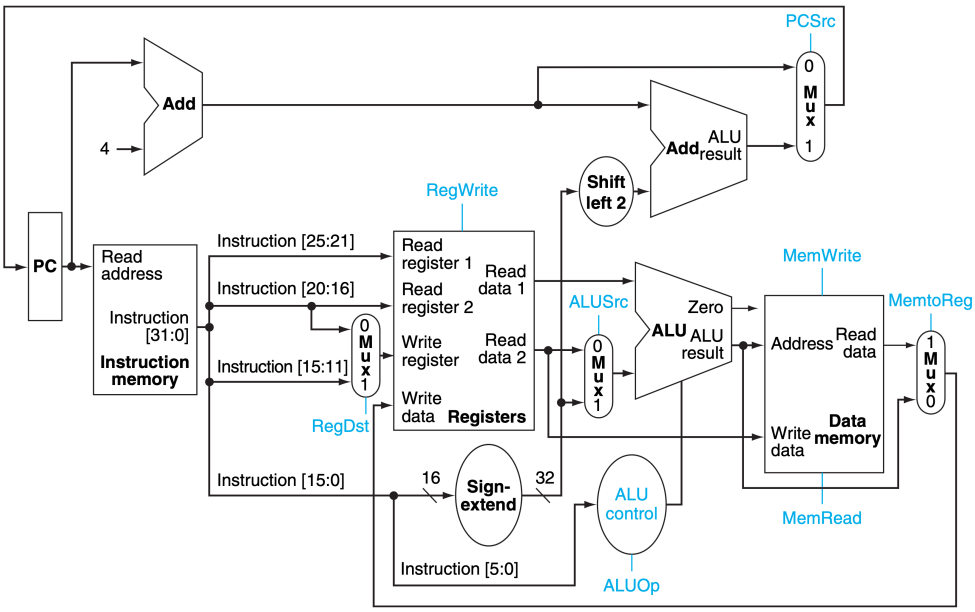
Only jal needs this path to write the next PC value into the register file so that jr can return to that address.

5. Why do we not have hazards in the single-cycle processor?

2013-Jan-Processor Control and Datapath-6

a. Every instruction finishes before the next one starts so the register file always has the right data
b. No instructions are executing in parallel so there are no conflicts between instructions
c. Each instruction has access to the whole processor while it is executing so there is no fighting for resources
d. All of the above

6. What is read out of Read data 2 in the register file for an I-format instruction?



Name	Bit Fields					
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Format	op	rs	rt	rd	shmt	funct
I-format	op	rs	rt	**address/immediate (16)**		
J-format	op	**target address (26)**				

2014-Apr-Processor Control and Datapath-4

a. Nothing
b. The immediate
c. The value stored in the register identified by the top 5 bits of the immediate value
d. For immediate ALU instructions (e.g. addi, etc.) the value stored in the register that this instruction will be writing to

The Read register 2 input is still 5 bits from the instruction, but now those bits happen to be from the rt field of the instruction. This means that Read data 2 outputs the register to be written to, or the value to be stored to memory (for sw instructions).

6. Pipelining

1. What is the purpose of pipelining a processor?
   `2023-Jun-Pipelining-1`
   a. To decrease the instruction latency
   b. To lower the clock frequency by split up stages
   c. To utilize different parts of the processor at the same time and do meaningful work
   d. To increase the effective capacity of memory

   Pipelining increases instruction latency but also increases instruction throughput. By splitting up the processor into smaller stages, the clock frequency can be *increased*. Utilizing different parts of the processor enables the higher throughput (answer). Pipelining does not increase the effective capacity of the memory. *Based on question contributed by student*

2. What does the pipeline do that causes problems with the ISA's promise of atomic execution?
   `2016-Jan-Pipelining-6`
   a. Nothing: there is no problem
   b. It executes parts of instructions across multiple stages
   c. It executes multiple instructions at the same time
   d. B and C

   Because instructions are split across multiple stages and multiple instructions are executed at the same time, we can end up with instructions requesting data that has not been written back.

3. Why do single-cycle processors have slow clock speeds?
   `2013-Dec-Pipelining-1`
   a. They have to handle a different length instructions
   b. They are limited by the slowest instruction
   c. They can't do any instructions in parallel
   d. All of the above

   While all of the above are true, it is only because they are limited by the slowest instruction that they have a slow clock speed. The others are complications (a) or performance problems (c) but they don't affect the clock speed directly.

4. What is the most important thing for getting performance from a pipelined design?
   `2013-Aug-Pipelining-6`
   a. Have as few pipeline registers as possible
   b. Make each stage fast
   c. Balance the latency of each stage
   d. Keep the pipeline full

   If you can't keep the pipeline full then the others don't matter.

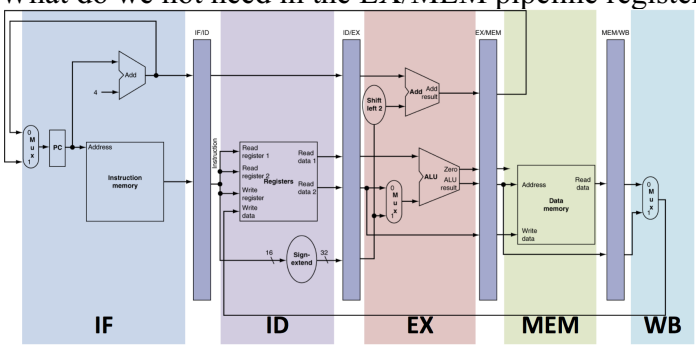
5. A processor is perfectly pipelined with 5 stages and runs at 5x the clock speed with no overhead, but the pipeline is only 80% full. What is the speedup over the original processor?

   `2017-Jan-Pipelining-2`

   a. 0.8x
   b. 4x
   c. 5x
   d. 20x

   If the pipeline was kept full we would finish instructions 5x faster. But since it's kept only 80% full, we get only 80% of this speedup, or 4x.

6. What do we not need in the EX/MEM pipeline register?



   `2013-Dec-Pipelining-5`

   a. ALU result
   b. Register read data 1
   c. Register read data 2
   d. Branch address

   We need the ALU result to send to the memory or for write back. We need Register read data 2 if we are going to do a write as that is the data to write. We need the Branch address since the branch is done in the MEM stage. We do not need Register Read data 1 because it has already been used as an input to the ALU.

7. Hazards

1. Given the following code, what is **not** correct about the instructions and hazards? Assume no double pumped registers and no data forwarding.

```
Start:
1: addi $t0, $r0, 30
2: addi $t1, $r0, 12
3: bne $t0, $t1, Start
4: add $t2, $t0, $t1
5: addi $t0, $t2, 10
```

   2023-Jun-Hazards-1
   a. Line 1 and 4 cause a structural hazard
   b. Line 3 causes a control hazards
   c. Line 1 and 3 cause a data hazard
   d. Line 3 and 4 cause a data hazard

   Lines 3 and 4 do not cause a data-hazard. `$t0` or `$t1` is only read by the `bne` and does not cause a data hazard. ***Based on question contributed by student***

2. How can forwarding help alleviate structural hazards?
   2016-Oct-Hazards-4
   a. By forwarding a value to the previous stage
   b. By forwarding a value to the next stage
   c. A and B
   d. Forwarding cannot alleviate structural hazards

   Structural hazards occur when the same hardware is needed by multiple instructions at the same time. Forwarding can only help with data hazards.

3. What is needed to resolve any hazards in this example?

```
addi R14, R14, 8
lw R1, 4(R13)
add R1, R1, R13
sw R14, 4(R13)
```

   2016-Aug-Hazards-4
   a. Forwarding from MEM to EX
   b. Forwarding from WB to MEM
   c. Can't resolve; need to stall
   d. There are no hazards to resolve

   The only dependencies are between the first addi and the last sw, but this is far enough to not require any forwarding, and between the lw and the add. However, the lw doesn't get its data back from the memory until after the MEM stage so there is no way to forward it to an instruction in the previous stage, so we cannot solve this with forwarding.

4. What is needed to resolve the following hazard?

```
xor R12, R13, R12
ld R2, R3
add R13, R4, R12
```

2014-Dec-Hazards-1

   a. Double-pumping the register file
   b. Forwarding from WB to EX
   c. Forwarding from MEM to EX
   d. There is no hazard to resolve, there are enough instructions between the dependencies.

The third instruction needs R12 in the EX stage, but the results from the previous instruction are only in the WB stage, so we need to forward from WB to EX.

5. In a certain program run on a 5-stage pipeline processor with double-pumped register file but no forwarding, 50% of all instructions are loads and 10% of the instructions directly following the loads depend on it. What is the slowdown of the program due to these dependencies?

2017-Jan-Hazards-6

   a. 1.0x (no slowdown)
   b. 1.05x
   c. 1.1x
   d. 1.2x

You will need 2 bubbles to handle the data dependency, so the dependent instructions will have a throughput of one-third IPC, or 3 cycles per instruction. 0.5*0.10*3+(0.5 + 0.5*(1-0.10))*1=1.1. (CPI of load & dependent + (CPI of non-loads + CPI of load & independent))

6. Are branch delay slots a good idea?

2016-Jan-Hazards-6

   a. Yes: the compiler can put useful instructions in them almost all the time
   b. Yes: people typically recompile their programs for new processors
   c. A and B
   d. No: all future processors have to emulate them to maintain compatibility

Compilers have a hard time filing too many branch delay slots and almost no one recompiles their software just because they get a new processor. The overhead of maintaining compatibility with branch delay slots for future processors makes them a bad decision..

8. Branch Prediction and Exceptions and Interrupts

1. What is correct about dynamic branch predictors?
   `2023-Jun-BP-1`
   a. Dynamic branch predictors learn branch patterns while running the program.
   b. Current systems do not use dynamic predictors but use static predictors instead
   c. A dynamic branch predictor always requires one bit per branch entry
   d. Dynamic branch predictors always perform better than static branch predictors

   Dynamic branch predictors learn based on dynamic runtime-branch behavior. Static predictors always predict in a static, predefined manner. Current systems mostly employ dynamic branch predictors. Dynamic branch predictors can have varying amount of data per branch entry (think 1-bit, 2-bit, etc. predictors. There are some cases where static branch predictors can be better: think a loop that loops back. This will only result in one miss for a BTFNT on exit, but two misses (one on initial miss, second on final jump out) on a 1-bit predictor. ***Based on question contributed by student***

2. What is prediction accuracy of a 2-bit branch predictor when running the following code?
```
for (i = 0; i < 1000; i++) // Loop 1000 times {
  if (i % 2 == 0) { //If value of i is even
    //Do something
  }
}
```
   Assume that the branch predictor begins with an initial state of strongly taken (11) and that the branch predictor can track multiple branches.
   Branch 1: `for` loop, Branch 2: `if` statement.
   `2023-Jun-BP-2`
   a. Branch 1: 997/1000, Branch 2: 500/1000
   b. Branch 1: 998/1000, Branch 2: 750/1000
   c. Branch 1: 998/1000, Branch 2: 500/1000
   d. Branch 1: 999/1000, Branch 2: 500/1000

   The branch predictor is initialized to strongly taken. Due to this, the for loop will only miss when jumping out at the very end. Thus 999/1001 for the for loop. For branch 2, we get 500/1000. ***Based on question contributed by student***

3. For a given program 80% of the branches are taken and our branch predictor is 75% accurate. A correct branch prediction costs 1 cycle and a misprediction costs 8 cycles. 12% of the executed instructions are branches. How much slower would the processor run (compared to a proeccor with a 100% accurate branch predictor). Assume that all other non-branch instructions have a CPI or IPC of 1.
   `2023-Jun-BP-3`
   a. 0.80x
   b. 0.88x
   c. 1.21x
   d. 1.84x

   80% branch taken is unnecessary information. 0.12 * (0.75*1 + 0.25*8) + 0.88 * 1 = 1.21 CPI ***Based on question contributed by student***

4. Why do we predict branches in modern processors?
   `2013-Dec-Branch Prediction and Exceptions and Interrupts-1`
   a. Can't afford the extra transistors to calculate the branch earlier
   b. The pipelines are long and we can't move the branch calculation earlier
   c. Branch predictors are faster than calculating the branch
   d. Predicting a branch is nearly perfectly accurate

   We couldn't move the branch calculation early enough in the simple MIPS 5-stage pipeline because we needed to access the register file. Modern processors have even more complex and longer pipelines, which makes this harder. We could afford the transistors if we could actually build it.

5. When an exception occurs, what happens to the other instructions in the pipeline?
   `2013-Aug-Branch Prediction and Exceptions and Interrupts-6`
   a. They are all killed
   b. They are killed if they come after the instruction with the exception
   c. They are killed if they have a data dependency on the instruction with the exception
   d. The operating system decides which instructions to kill

   The instructions after the instruction with the exception have to be killed because we need to handle the exception before continuing.

6. If we add a branch target buffer (BTB) to the branch predictor, do we still need an adder to calculate the branch address?
   `2014-Sep-Branch Prediction and Exceptions and Interrupts-6`
   a. Yes, but only if we predict the branch incorrectly
   b. Yes, to verify that the target was correct even if we predict correctly
   c. Only the first time we see the branch so we can store the address in the BTB
   d. No, the instruction provides the address

   We still need to calculate the address to know if the address in the BTB was correct. This is particularly true for instructions where the address is not a constant, such as JR.

9. Input/Output

1. What are the benefit(s) of polling over interrupt-based I/O?
   `2016-Aug-Input/Output-1`
   a. Polling can detect ready data sooner
   b. Polling uses less processor time
   c. Polling has higher throughput for large data transfers
   d. All of the above

   Polling can detect that data is ready sooner because that's all the processor is doing. Once it detects the data is ready, the transfer is the same rate as interrupt-based I/O and the cost of doing this is more processor time spent checking if the data is ready.

2. Why don't we use DRAM for data storage instead of disks and flash?
   `2013-Jan-Input/Output-1`
   a. Too expensive
   b. Too slow
   c. Too unreliable
   d. Something else

3. How do DMA and memory-mapping interact?
   `2015-Apr-Input/Output-5`
   a. DMA cannot be used if the device is memory-mapped
   b. DMA uses the device's interrupt to move data
   c. DMA moves data by accessing the device directly
   d. DMA transfer is set-up using memory-mapped I/O

   DMA and memory-mapped IO are orthogonal. DMA engines can be configured/setup using memory-mapped I/O. The DMA avoids having to have a program running on the CPU take care of the copy.

4. Why do serial communications send two versions of the same signal?
   `2017-Jan-Input/Output-4`
   a. To check against each other to make sure the final signal is correct
   b. To subtract from each other to cancel out noise that affects both signals
   c. To reduce energy by sharing noise between the wires
   d. All of the above

   The two signals allow the effect of common noise to be eliminated by subtracting the signals from each other.

5. Rank the following technologies in terms of cost per MB: flash, DRAM, hard disk, tape
   `2013-Aug-Input/Output-2`
   a. (cheapest) flash < DRAM < hard disk < tape
   b. (cheapest) tape < hard disk < DRAM < flash
   c. (cheapest) tape < DRAM < flash < hard disk
   d. (cheapest) tape < hard disk < flash < DRAM

   Tape is the absolute cheapest (just buy more tapes), hard disks are next cheapest, and then flash. DRAM is 10x more expensive than flash.

6. What is true about the following?
   `2023-Jun-IO-1`
   a. Recently spinning HDD performance has improved greatly
   b. The performance of SSDs has forced the industry to introduce a new I/O protocol.
   c. As the speed of disks increased, the relative software overhead became smaller.
   d. With faster disks, it is more favorable to use interrupt, not polling.

   HDDs have been stable in performance. SSDs became too fast for existing SATA protocol, resulting in the NVMe protocol. As disk speed increased, the SW overhead became larger. Faster disks benefit from polling, not interrupt.

10. Caches

1. What is true about dirty bits in caches?
   `2023-Jun-Caches-1`
   a. Cache lines with dirty bits need to eventually be written back to the main memory (DRAM)
   b. Dirty bits are required for write-through caches
   c. A dirty bit shows that the data has changed in the memory (DRAM), but not in the cache yet.
   d. Bits loose voltage over time which can corrupt data. Dirty bits are set for such data that have been corrupted/dirtied

   Dirty bits state that data in cache has been updated (but not in main memory yet). Dirty bits concern write-back caches. SRAMs (caches) don't lose voltage over time, DRAMs do. Thus DRAMs need refreshes. ***Based on question contributed by student***

2. Which of the following types of caches does not need a replacement policy?
   `2023-Jun-Caches-2`
   a. Direct-mapped caches
   b. Set-associative caches
   c. Fully-associative caches
   d. All types of caches need replacement policies

   Direct mapped caches do not need replacement policies because there is only one block in the set, meaning if another cache line is inserted, there is only one (obvious) line that can be evicted. ***Based on question contributed by student***

3. What is true about large cache blocks (or cache lines)?
   `2023-Jun-Caches-3`
   a. Bigger blocks result in more bits used in the tags (wasting space)
   b. Bigger blocks are not effective for sequential data accesses
   c. Bigger blocks provide higher spatial locality
   d. Real processors tend to have very small cache blocks (on or two words)

   Spatial locality is improved by larger blocks. Bigger blocks are well suited for sequential data access. Furthermore, larger blocks result in less number of blocks, resulting in less bits in the tags. Finally, real processors use 8 words or even 16 word blocks (64B, 128B). ***Based on question contributed by student***

4. What is **not** true about the following:
   `2023-Jun-Caches-4`
   a. Tag bits in the address are used to verify whether the data in the cache is indeed the data that we want to access/modify.
   b. Index bits in the address are used to locate which way of the cache to lookup.
   c. Index bits are not required for fully-associative caches
   d. Fully-associative caches require more HW logic than direct-mapped caches.

   Index bits in the address are used to locate which *set* to lookup.

5. Assume a 4-entry fully-associative caache with one-byte per line, and a LRU replacement policy. How many misses occur for the following accesses?
   `0, 1, 2, 3, 4, 5, 3, 2, 1, 2, 5`
   `2023-Jun-Caches-5`
   a. 6
   b. 7
   c. 8
   d. 9

   First six accesses cause 6 compulsory misses. The access to '1' (third to final access) causes a capacity miss.

6. Why do we have a memory hierarchy?
   `2013-Apr-Caches-1`
   a. To make the memory look big
   b. To make the memory look fast
   c. Because we can't afford tons of SRAM
   d. All of the above

   The small cache is fast for recent data, and the large memory is large for lots of data. SRAM would be great instead of DRAM, but it's too expensive and too high power.

11. Virtual Memory

1. Writing to which address with the following page table would result in a page being loaded from disk?

```
VA-->PA          on disk          access bits
2-->8            0                read/copy on write
1-->9            0                read/write
14-->7           0                read/write
16-->14          1                read/write
13-->6           1                read only
```

   2015-Apr-Virtual Memory-3
   a. 8
   b. 13
   c. 14
   d. 16

   Writing to address 8 would result in memory protection exception/segmentation fault since that address is not in the page table. Address 13 would result in a segmentation fault (since you don't have permission), address 14 would result in a direct write to DRAM since the page is in memory, but address 16 requires that the data be loaded from disk before it can be written.

2. Two programs want to share data. Can this be done with virtual memory?
   2013-Aug-Virtual Memory-6
   a. No, each program has its own virtual address space that maps to its own physical memory.
   b. Yes, they can both have pages that map to the same physical memory.
   c. Only if they use a physically-index and physically-tagged cache.
   d. Only if they use a virtually-index and virtually-tagged cache.

   Programs can share data by having the OS map their virtual addresses to the same physical page. This does cause problems for the cache, but with a combination of flushing and process ID tags these can be overcome.

3. What are the benefits of virtual memory?
   2015-Oct-Virtual Memory-3
   a. It increases security by isolating each process' memory
   b. It frees the applications from having to manage a shared memory
   c. It allows an application to use more memory than physically available
   d. All of the above

   Virtual memory gives the illusion to each program that it has sole access to the whole address space. The advantages are multiple: It increases security by preventing an application from accessing another program's memory. It also frees applications from having to manage a shared physical memory (this burden is on the OS instead). Finally, each applications has access to the whole virtual address space, even if there is not enough physical memory available. In this case, the OS will use secondary storage transparently.

4. Computers typically use 4kB pages, but often support larger (2MB and 1GB) pages as well. What is the advantage of larger pages for programs that access data sequentially?
   `2016-Jan-Virtual Memory-3`
   a. Fewer TLB misses
   b. More TLB misses but faster DRAM access
   c. Can access larger regions of memory in the program
   d. All of the above

   Larger pages allow the program to access more data for each TLB entry, which reduces the number of TLB misses..

5. Why is a Virtually Indexed, Physically Tagged cache a good design?
   `2016-Aug-Virtual Memory-6`
   a. You can access the TLB and the cache tags in parallel to save time.
   b. The cache index address is unique across programs to provide security.
   c. Using separate types of addresses allows the TLB and cache to be smaller so they are faster.
   d. All of the above.

   VIPT allows us to do the translation (VA->PA) a the same time as we look up the tag (VA->PA tag) and then after both happen in parallel we can compare them. This avoids having to do the VA->PA TLB translation first before we can do a PA->PA tag lookup..

6. Why do we have multi-level page tables?
   `2013-Dec-Virtual Memory-4`
   a. To reduce the maximum size of the page table
   b. To reduce the in-memory size of the page table
   c. To make the TLB faster
   d. All of the above

   Multilevel page tables let us page out part of the page table as we can find it again later from the first-level page. This reduces the size of the in-memory page table.

12. Parallelism

1. Why do we do parallelism?
   `2017-Jan-Parallelism-5`
   a. Faster
   b. Easier
   c. Lower power
   d. All of the above

   Parallel code and hardware is (a lot) harder than serial code and hardware. Parallel execution can be more power-efficient than serial, but it doesn't in itself reduce power. We only put up with the pain and complexity of parallelism for performance.

2. How do locks prevent problems with multiple processors accessing data at the same time?
   `2016-Jan-Parallelism-3`
   a. They tell the hardware to prevent multiple accesses to the data
   b. They tell the hardware to keep the data synchronized across multiple processors
   c. They tell the software when it's processor is allowed to access the data
   d. They allow the software to coordinate which processor accesses the data

   Locks allow programmers to check if another processor has requested access to some data and request access. It is up to the software to both check and then only access the data if it gets access, as the lock itself does not do anything to enforce synchronization.

3. What happened in 2003/2004 that led us to have multicore chips everywhere today?
   `2017-Jan-Parallelism-6`
   a. Processors got small enough that you could put multiple cores on the same chip to get better performance
   b. Processors got so hot that it was too expensive to cool them, so manufacturers switched to putting multiple slower (cooler) processors on a chip
   c. Manufacturers couldn't increase the clock speed, so they put multiple cores on the same chip to get better throughput
   d. All of the above

   Processors never really got smaller: people kept adding more and more features so they were getting larger even faster than the rate at which transistors were getting smaller. Manufacturers could indeed increase the clock speed (Intel did). The killer was power: it was too expensive to cool faster chips so they switched to multicore and we've been suffering ever since.

4. How much faster can my program run if I have 9000 cores and 10% of my program cannot be parallelized?
   `2017-Jan-Parallelism-4`
   a. 9000x faster
   b. 4500x faster
   c. 5x faster
   d. 10x faster

   Using Amdahl's law, we get a speedup of 1 / (0.10 + 1/9000 * 0.90) = 10x

   Another way to answer this question is to say that by having infinite parallelism, 90% of this program will be done in 0 unit of time. Only 10% of it must be done serially, and thus the total execution time is divided by 10..

5. What do you **not** need to add to the pipeline to support executing load/store instructions in parallel with other (non-load/store) instructions?
   `2013-Jan-Parallelism-3`
   a. Extra ALU
   b. Extra data memory ports
   c. Extra register file ports
   d. Extra instruction memory ports

   One part of the execution will be exclusively doing memory instructions, the other, all-other instructions. Thus we only need one set of data-memory ports. Everything else is needed, ALU (now need to generate address and do other ALU ops at the same time), register-file-port (need to read another set of data/address, instruction (need to read another instruction because we are going to a 2-issue processor).

6. Which of the following processor designs can execute more than one instruction at the same time?
   `2018-Oct-Parallelism-1`
   a. Pipelined Single-issue
   b. Pipelined Dual-issue
   c. Pipelined Symmetric Multithreading (SMT)
   d. All of the above

   A pipelined processor processes multiple instructions at the same time across the stages (pipeline parallelism). Dual-issue means it can do twice as many at the same time. SMT, however, just means it can switch between instructions from different programs, but it may or may not do them at the same time.

Make sure you circle the answers on the answer page.
Answers on other pages will NOT be graded.